

Hierarchical DBMS

Data models in database systems

Lecture 9

Mara Romanovska
Mara.Romanovska@rtu.lv

04.12.2025.



RTU
DATORZINĀTNES UN
INFORMĀCIJAS
TEHNOLOĢIJAS FAKULTĀTE

Agenda

- PW3
- MongoDB as one of the options for native storage
- MongoDB data model
- MongoDB architecture
- Data querying with MongoDB

PW3: Design and Implementation of a Data Warehouse

- The submission deadline is **December 28**
 - An emergency extension may be requested via email before the deadline if you have not used it before for PW1 or PW2
- PW3 is a **an individual project**
- Tasks:
 - Import JSON documents into MongoDB
 - Perform transformations
 - Design and run pipelines
- Description and variation generation published by tomorrow

NB!

- Lecture omits installation and data import as those are fairly simple steps with plenty tutorials
- You will have to demonstrate them for PW3

MongoDB in general

- MongoDB has evolved out of a necessity for flexible schema database
- Started as an open source project in 2007
- Rebranded from 10gen to MongoDB in 2013 and was released in 2017
- Typical document database: «data that is accessed together, is stored together», so **data replication is encouraged**

JSON vs. BSON

- BSON is binary JSON encoding
- Mainly used in MongoDB
- Everything is stored in BSON format but formats are interchangeable

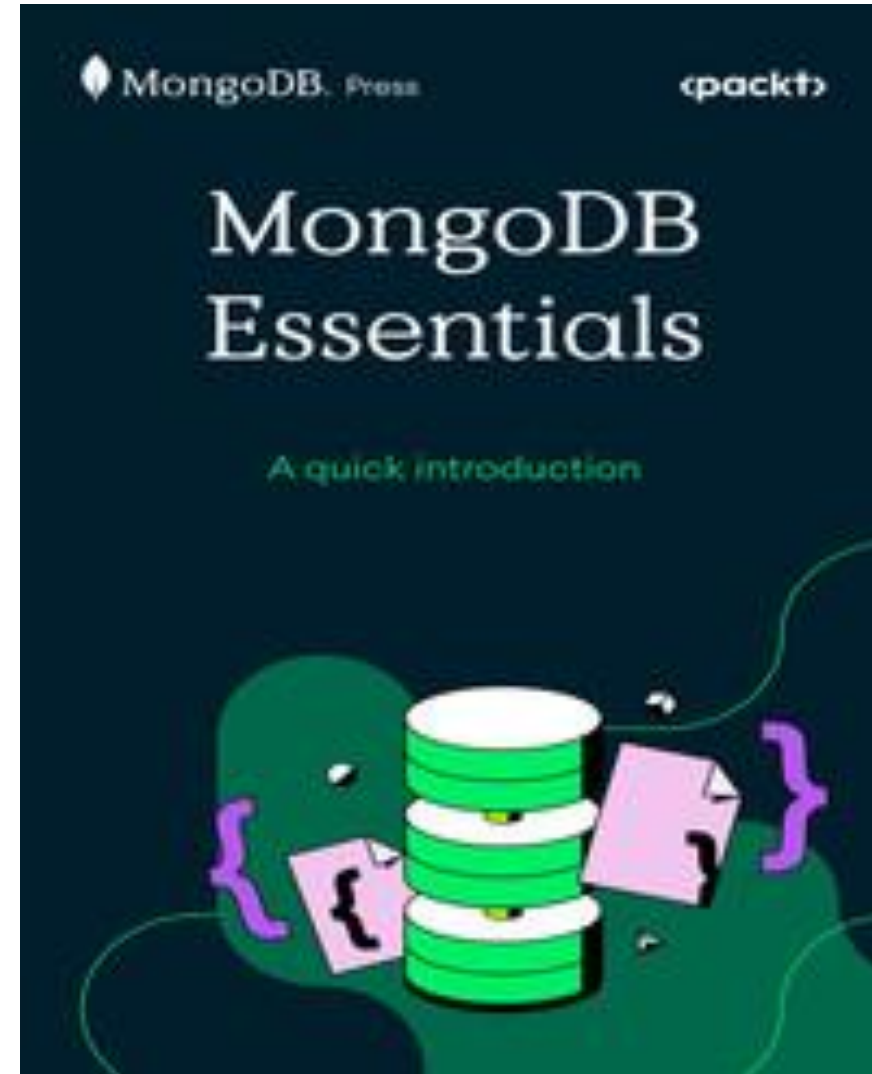
```
1 {"hello": "world"} →  
2 \x16\x00\x00\x00 // total document size  
3 \x02 // 0x02 = type String  
4 hello\x00 // field name  
5 \x06\x00\x00\x00world\x00 // field value  
6 \x00 // 0x00 = type E00 ('end of object')
```

<https://www.mongodb.com/resources/basics/json-and-bson>

Accessible via ORTUS e- resources...

...but plenty materials exist on website

Note that this book is developed by MongoDB Team – so evaluate any praises of MongoDB critically



Data model



Data model in general

- MongoDB uses document data model
- The structure can be **roughly** translated as
 - Documents = records
 - Collections = tables
 - Fields = attributes
- Merges flexible schema with querying indexing of RDBMSs
- No need to explicitly define schemas or structures upfront (typical for native storage)

Main principles

- **Polymorphism** – documents can have multiple schemas in the same collection
- **Arrays** – difference from XML
- **Embedding** – i.e., nesting
- **Referencing** when necessary

Documents

- JSON-like structures stored internally in **Binary JSON (BSON)**
- Documents are **self-contained** units of information
- Ideally represent single object or entity in your app
- Documents can have embeddings aka subdocuments aka nested documents

JSON Schema

- MongoDB supports **schema validation**
- Using **JSON Schema**, you can enforce the presence of fields, constrain data types, or define nested document structures – similar to XML Schema

Collections

- Next organizational structure after documents
- Note there are no «root element», just «a collection»
- Collections group **related documents** with an **optional schema**
- Collections are put into **databases**, which provide logical separation as MongoDB deployment can have multiple databases

```
|  _id: ObjectId('639f639ab0a64cc36d4f7802')  
  : "0"  
  ▶ track: Object  
    track_id: "4LRPiXqCikLlN15c3yImP7"  
    id: "6KImCVD70vtIoJWnq6nGn3"  
    name: "Harry Styles"  
    listened_times: "271,866,522"  
    artists_country: "UK"
```

```
  _id: ObjectId('639f639ab0a64cc36d4f7803')  
  : "1"  
  ▶ track: Object  
    track_id: "3rmo8F54jFF80gYsqTxm5d"  
    id: "6eUKZXaKkcviH0Ku9w2n3V"  
    name: "Ed Sheeran"  
    listened_times: "1,048,851,840"  
    artists_country: "UK"
```

```
  _id: ObjectId('639f639ab0a64cc36d4f7804')  
  : "2"  
  ▶ track: Object  
    track_id: "3gdewACMIVMEWVbyb809sY"  
    id: "3PhoLpVuITZKcymwpck5b"  
    name: "Elton John"  
    listened_times: "709,072,740"  
    artists_country: "UK"
```

Indexes

- A default **_id index** on every collection
- Support for indexing any field, including nested fields and array values
- There is a specialized B-tree-like structure for storing indexes

```
_id: ObjectId('639f639ab0a64cc36d4f7802')  
: "0"  
▶ track: Object  
  track_id: "4LRPiXqCikLLN15c3yImp7"  
  id: "6KImCVD70vtIoJWnq6nGn3"  
  name: "Harry Styles"  
  listened_times: "271,866,522"  
  artists_country: "UK"
```

```
_id: ObjectId('639f639ab0a64cc36d4f7803')  
: "1"  
▶ track: Object  
  track_id: "3rmo8F54jFF80gYsqTxm5d"  
  id: "6eUKZXaKkcviH0Ku9w2n3V"  
  name: "Ed Sheeran"  
  listened_times: "1,048,851,840"  
  artists_country: "UK"
```

```
_id: ObjectId('639f639ab0a64cc36d4f7804')  
: "2"  
▶ track: Object  
  track_id: "3gdewACMIVMEWVbyb809sY"  
  id: "3PhoLpVuITZKcymwpck5b"  
  name: "Elton John"  
  listened_times: "709,072,740"  
  artists_country: "UK"
```

Dealing with JOINS

- There are no joins, instead there are two mechanisms:
 - **Embedding** related data directly inside documents
 - **Referencing** documents by their unique `_id`
- Embeddings and referencings are used to implement **relationships**

Embeddings vs. referencing

- Embeddings used when:
 - Related data is always or usually **accessed with the parent**
 - Embedded content **doesn't grow unbounded**
 - Data is **specific** to that document and not shared elsewhere
- Referencing is used when:
 - Related data is **large or frequently updated independently**
 - Related data is **reused across multiple documents**
 - You want to prevent a single document from growing too large

Denormalization

- Denormalization is a common and encouraged strategy in DDBSs, including MongoDB
- Done through embeddings
- You can also **embed arrays**

Relationships

- One to one is implemented through embedding
- One to many:
 - Many side is embedded directly into one side
 - Referencing subdocument with unique keys
- Many to many:
 - Again, embedding or referencing

```
_id: ObjectId('639f639ab0a64cc36d4f7802')
: "0"
track: Object
  album: Object
    release_date: "3/31/2022"
    available_markets: "['AD', 'AE', 'AG', 'AL', 'AM', 'AO', 'AR', 'AT', 'AU', 'AZ', 'BA', 'BB...]"
    name: "As It Was"
    popularity: "100"
  track_id: "4LRPiXqCikLLN15c3yImp7"
  id: "6KImCVD70vtIoJWnq6nGn3"
  name: "Harry Styles"
  listened_times: "271,866,522"
  artists_country: "UK"
```

MongoDB architecture

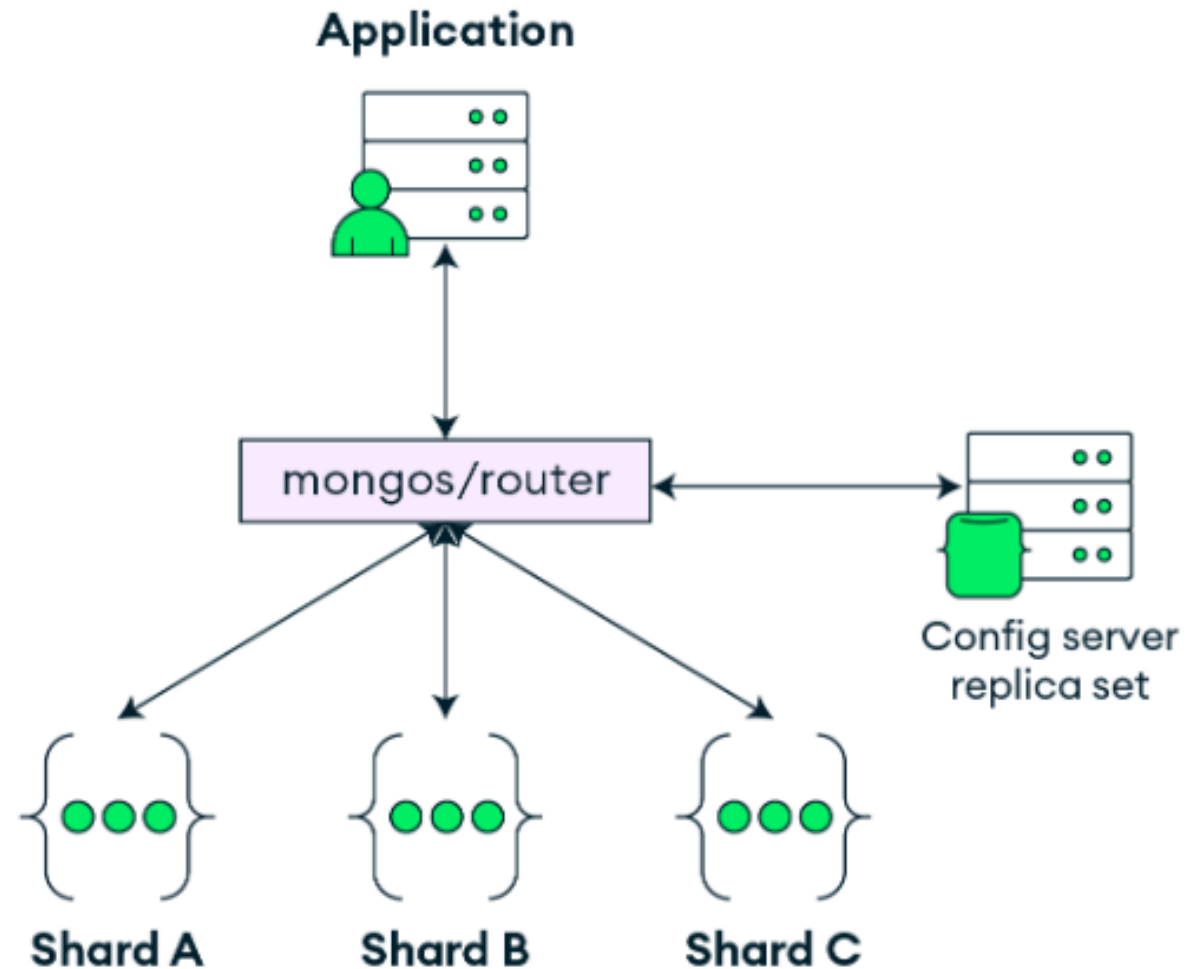


Storage engine and query execution

- Every MongoDB instance includes three components:
 - The query planner
 - The execution engine
 - The storage engine
- MongoDB also have replica sets – this enables automatic recovery
- This enables MongoDB to be ACID-compliant

Sharding: scaling horizontally

- MongoDB splits your data across multiple machines based on a **shard key**, a field that determines how documents are distributed



Indexing documents

- There is an automatic index created on `_id` field
- There can be compound indexes created
- Indexes can be unique

CRUD in MongoDB



CRUD operations

- Queries are expressed using JSON-style syntax
- For a change of pace, are not SQL-like

Inserting documents

- Can be done via importing or `.insertOne/.insertMany` commands
- MongoDB automatically assigns a unique **`_id` field** to each document if you don't provide one

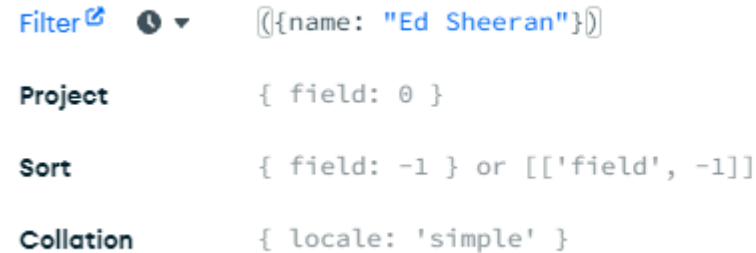
```
_id: ObjectId('639f639ab0a64cc36d4f7802')  
  : "0"  
▶ track: Object  
  track_id: "4LRPiXqCikLLN15c3yImP7"  
  id: "6KImCVD70vtIoJWnq6nGn3"  
  name: "Harry Styles"  
  listened_times: "271,866,522"  
  artists_country: "UK"
```

Updating and deleting

- For updating: updateOne, updateMany, replaceOne
- For deleting: DeleteOne, deleteMany

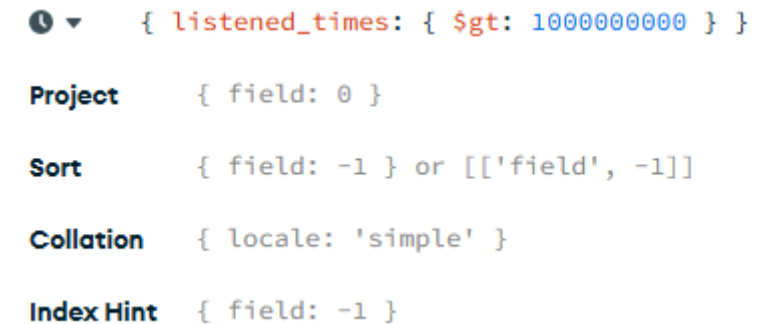
Filtering documents

- `db.books.find({name: "Ed Sheeran"});`



The screenshot shows the MongoDB Compass interface. At the top, there is a 'Filter' button with a link icon and a clock icon with a dropdown arrow. To the right of these icons is the filter query: `{name: "Ed Sheeran"}`. Below this, there are four configuration rows: 'Project' with `{ field: 0 }`, 'Sort' with `{ field: -1 } or [['field', -1]]`, and 'Collation' with `{ locale: 'simple' }`. The 'Index' row is not visible.

- Operator – what you are looking for
 - Logical operators such as `$and`, `$or`, and `$not`
 - Comparison operations such as `$gt`, `$lt`, and `$in`
 - Access to nested fields using dot notation (e.g., `"publisher.name"`)
 - Array-specific queries using operators such as `$elemMatch`



The screenshot shows the MongoDB Compass interface with a complex query. At the top, there is a clock icon with a dropdown arrow, followed by the query: `{ listened_times: { $gt: 1000000000 } }`. Below this, there are four configuration rows: 'Project' with `{ field: 0 }`, 'Sort' with `{ field: -1 } or [['field', -1]]`, 'Collation' with `{ locale: 'simple' }`, and 'Index Hint' with `{ field: -1 }`.

- Not in Compass might look like this: `db.books.find().sort({ publishedYear: -1 }).limit(5);`

Aggregations

- Documents go through data transformation **pipeline**
- An aggregation pipeline provides a structured way to transform, combine, and analyze documents within a collection
- Stages are **chained** – i.e., one ends, the other begins
- Uses a **JSON-like** notation

Core building blocks

- **Stages**
 - \$match, \$group, \$sort, and \$project
 - define what each step in the pipeline does
- **Accumulator expressions**
 - \$sum, \$avg, \$push, and \$max
 - used inside grouping stages to compute values across grouped documents
- **Path expressions** such as "\$field" or "\$author.name" let you reference fields in your documents
- **Conditional expressions**
 - \$cond, \$ifNull, and \$switch
 - support logic-based transformations within the pipeline
- **System variables**

Aggregation stages

- You define a series of **stages**
- **\$match** stage to filter documents
- **\$group** stage to aggregate data
- **\$sort** stage to order the results
- **\$project** stage to reshape the output
- The sequence matters!

Changing types in the document with aggregation

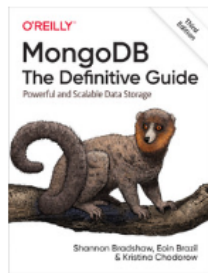
▼ Stage 1 \$set 

```
1 {  
2   listened_times: {  
3     $convert: {  
4       input: "$listened_times",  
5       to: "double"  
6     }  
7   }  
8 }
```

▼ Stage 2 \$merge 

```
1 {  
2   into: "Playlists"  
3 }
```

Domain



MongoDB: The Definitive Guide, 3rd Edition

[Shannon Bradshaw](#), [Eoin Brazil](#), [Kristina Chodorow](#)

Published by [O'Reilly Media, Inc.](#)

```
{
  "_id" : "52cdef7c4bab8bd675297d8e",
  "name" : "Facebook",
  "category_code" : "social",
  "founded_year" : 2004,
  "description" : "Social network",
  "funding_rounds" : [{
    "id" : 4,
    "round_code" : "b",
    "raised_amount" : 27500000,
    "raised_currency_code" : "USD",
    "funded_year" : 2006,
    "investments" : [
      {
        "company" : null,
        "financial_org" : {
          "name" : "Greylock Partners",
          "permalink" : "greylock"
        },
        "person" : null
      },
      {
        "company" : null,
        "financial_org" : {
          "name" : "Meritech Capital Partners",
          "permalink" : "meritech-capital-partners"
        },
        "person" : null
      }
    ],
    "person" : null
  },
  {
    "company" : null,
    "financial_org" : {
      "name" : "Founders Fund",
      "permalink" : "founders-fund"
    },
    "person" : null
  }
],
}
```

```
{
  "company" : null,
  "financial_org" : {
    "name" : "SV Angel",
    "permalink" : "sv-angel"
  },
  "person" : null
},
{
  "id" : 2197,
  "round_code" : "c",
  "raised_amount" : 15000000,
  "raised_currency_code" : "USD",
  "funded_year" : 2008,
  "investments" : [
    {
      "company" : null,
      "financial_org" : {
        "name" : "European Founders Fund",
        "permalink" : "european-founders-fund"
      },
      "person" : null
    }
  ],
  "person" : null
},
{
  "ipo" : {
    "valuation_amount" : NumberLong("104000000000"),
    "valuation_currency_code" : "USD",
    "pub_year" : 2012,
    "pub_month" : 5,
    "pub_day" : 18,
    "stock_symbol" : "NASDAQ:FB"
  }
},
}
```


Pipeline examples (1)

```
db.companies.find({founded_year: 2004})
```

```
db.companies.aggregate([  
    {$match: {founded_year: 2004}},  
    ])
```

```
db.companies.aggregate([  
    {$match: {founded_year: 2004}},  
    {$project: {  
        _id: 0,  
        name: 1,  
        founded_year: 1  
    }}  
    ])
```

```
{"name": "Digg", "founded_year": 2004 }  
{"name": "Facebook", "founded_year": 2004 }  
{"name": "AddThis", "founded_year": 2004 }  
{"name": "Veoh", "founded_year": 2004 }
```

Pipeline examples (2)

```
db.companies.aggregate([
  {$match: {founded_year: 2004}},
  {$project: {
    _id: 0,
    name: 1}},
  {$limit: 5}
])
```

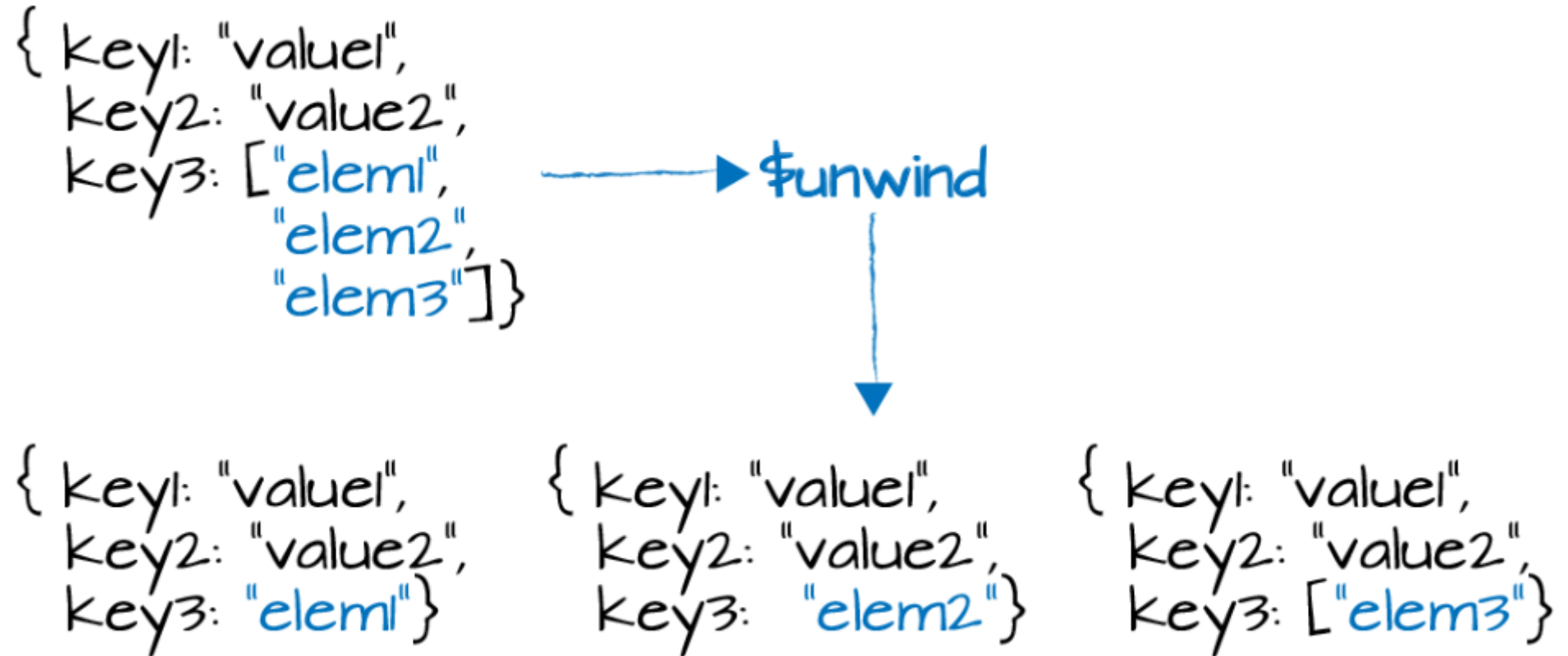
```
db.companies.aggregate([
  { $match: { founded_year: 2004 } },
  { $sort: { name: 1 } },
  { $limit: 5 },
  { $project: {
    _id: 0,
    name: 1 } }
])
```

```
db.companies.aggregate([
  {$match: {founded_year: 2004}},
  {$sort: {name: 1}},
  {$skip: 10},
  {$limit: 5},
  {$project: {
    _id: 0,
    name: 1}},
])
```

Reshaping with \$project

```
db.companies.aggregate([
  {$match: {"funding_rounds.investments.financial_org.permalink": '
  {$project: {
    _id: 0,
    name: 1,
    ipo: "$ipo.pub_year",
    valuation: "$ipo.valuation_amount",
    funders: "$funding_rounds.investments.financial_org.permalink"
  }}
]).pretty()
```

\$unwind



\$unwind

```
db.companies.aggregate([
  { $match: {"funding_rounds.investments.financial_org.permalink":
  { $unwind: "$funding_rounds" }},
  { $project: {
    _id: 0,
    name: 1,
    amount: "$funding_rounds.raised_amount",
    year: "$funding_rounds.funded_year"
  } }
])
```

```
{"name": "Digg", "amount": 8500000, "year": 2006 }
{"name": "Digg", "amount": 2800000, "year": 2005 }
{"name": "Digg", "amount": 28700000, "year": 2008 }
{"name": "Digg", "amount": 5000000, "year": 2011 }
```

Improving performance

- How you build aggregation pipelines impacts performance?

«

- **Filter early** with \$match and \$limit to reduce the volume of data passed through the pipeline
- **Use indexes** on fields involved in \$match or \$sort stages to speed up retrieval
- **Avoid unnecessary reshaping** with early \$project stages unless they help reduce payload
- **Pair \$sort with \$limit** for efficient top-N queries
- **Work with arrays smartly**; use \$map, \$filter, or \$reduce instead of always relying on \$unwind

»

ESR guideline

- A **query shape** is the combination of filters, sorts, and projections used in a query (note that it is not strictly defined)
- MongoDB matches indexes based on **shape**, not just fields
- To design compound indexes that match your query shapes, follow the **ESR guideline**:
 - **E**quality fields first
 - **S**ort fields next
 - **R**ange fields last

Connectivity with other apps

- MongoDB returns the document as a native Python dictionary
- No **object relational mapper (ORM)** is required
- MongoDB also allows connecting with intelligent apps

Thank you!

